

Distributed, Workflow-Driven Rendering of 3D Object Scenes on a Big Data Processing Platform

Constantin Nandra

Computer Science Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
constantin.nandra@cs.utcluj.ro

Victor Bacu

Computer Science Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
victor.bacu@cs.utcluj.ro

Dorian Gorgan

Computer Science Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
dorian.gorgan@cs.utcluj.ro

Abstract—This paper aims to explore and experiment the use of a general purpose, distributed processing solution for the rendering of 3D object scenes. Although originally designed for the processing of Earth Observation data, the platform was built with flexibility and extensibility in mind. At its core, it is based on the principle of task modularity, aiming to both simplify the specification of processing algorithms and decrease their overall execution time. To this end, it relies on the dynamic generation and distribution of sub-tasks within a distributed processing environment. Throughout the paper, we will be demonstrating its potential for rendering both still frames and videos of complex object scenes, by setting up, in effect, an ad-hoc rendering farm. This will help to highlight both the platform's utility in exploiting inherent task and data parallelism in order to improve execution time and its great flexibility and adaptability to different usage scenarios, and to a variety of processing operations.

Keywords—3D rendering; distributed processing; workflow model.

I. INTRODUCTION

Rendering high quality graphics has always been a computationally intensive task, so much so that it has led to the development of specialized hardware which, due to ever-increasing public demand, found its way into most computers nowadays. The Graphics Processing Unit or GPU is a ubiquitous component of virtually every personal computer, being used for a variety of purposes: from the running of the newest, most realistic and responsive games to allowing for scientific visualization and simulations. GPU development has been particularly well placed to take advantage of the development in semiconductor technologies that have occurred over the past few decades. The frequency limitation had something of a dislodging effect on the development of faster, more effective processors, forcing the shift of paradigm from the powerful single core chips to multiple processing cores – somewhat limited speed wise – situated on the same chip. The GPU, on the other hand, stood to benefit from this paradigm shift. This is because, unlike most traditional processing tasks which are linear, graphics processing is highly parallel in nature. This trend of moving towards parallel computing architectures fitted well with the processing necessities of the GPU, leading to major developments in terms of speed and efficiency within the last decade.

When talking about graphics rendering, parallelism is the key. Throughout this paper we will be emphasizing data parallelism, or the possibility of computing multiple subsets of the result simultaneously. There are multiple approaches to exploiting data parallelism – as described in [1]. Based on what gets distributed before the actual computation, there are two main approaches: sort first – which distributes the objects to be rendered – and sort last – which assigns pixel areas to be rendered. This paper will detail the usage of an existing processing platform – designed for dealing with Earth Observation data – for the purpose of rendering 3D scenes using a sort-last approach.

II. RELATED WORKS

Graphics rendering as an activity is highly parallelizable in its nature, since image portions and different frames can be rendered independently of one another. This is the main idea behind the concept of a render farm – a collection of processing nodes being fed the model to render and then being harvested for the rendered images or image portions. While the setup and running of such a render farm would certainly require resources not available to most users, with the advent of free and open-source 3D modelling suites such as Blender [2], the idea of the shared, common render farm started to take root. The concept is based on the end user's willingness to share their spare computing power in order to contribute to collaborative rendering efforts. In return, the user gets the opportunity to render their own work using the network's pooled computing resources.

One example of such a collaborative rendering platform is the BURP (Big Ugly Render Project) initiative [3]. It provides free software suite relying on the BOINC (Berkley Open Infrastructure for Network Computing) [4] middleware in order to distribute the processing tasks to its users. Other notable uses of BOINC include SETI@home - a volunteer computing project trying to scan the radio wave spectrum for signals of extraterrestrial origin. BOINC's most notable feature, however, is that it allows distributed applications to make use of GPUs, which makes it relevant in the context of participative rendering efforts. Another interesting example of free, open source distributed rendering software is Flamenco. It works in conjunction with Blender's Cycles renderer and is able to

dispatch a variety of processing tasks like rendering, encoding, compositing and simulation [5].

Considering the potential costs derived from operating an in-house render farm, most studios tend to use cloud-based render farm services. Among these one could count solutions such as Fox Render Farm [6], Render Rocket [7] and Rebus Farm Render [8]. They tend to offer commercially viable alternatives to the often prohibitive cost of running on-premises render farms.

The authors of [9], describe a HPC solution for volume rendering using heterogeneous processing resources and following a sort-last approach. It strives to provide a scalable processing solution, taking care of the networking aspects which might otherwise restrict its user base. This is indeed a concern apparent in the case of rendering tools which provide networked rendering capabilities, such as Blender [2]. Being able to properly set up and work with such a feature requires some level of skill or tool familiarity from the user's part.

An example of distributed GPU processing is offered by [10]. The paper presents a cluster solution designed to facilitate the remote interaction with and visualization of high-resolution simulations of particle-based systems.

Another interesting development in parallel rendering is presented within [11]. It details the usage of a framework called ButterFly, designed to migrate rendering workloads within a network of GPU-equipped mobile devices in order to improve rendering time and save on battery usage. The paper also tries to evaluate the user experience of offloading both qualitative and quantitative frame rendering over a Wi-Fi network.

III. DISTRIBUTED RENDERING SOLUTION

The processing solution proposed within this paper was originally meant to tackle the problem of handling large amounts of Earth Observation (EO) data received from ground, air and orbital sources. Also referred to by the name BigEarth [12], its design was centered on the principle of simplifying the specification of data processing algorithms meant to be executed within a distributed, parallel environment – Fig. 2. As it is addressing the needs of the typical earth data scientist, presumed to have little in the way of programming experience, the processing platform did away with the employment of complex parallel programming APIs, and opted instead for the use of a simplified representational model. This model was intended to help leverage the potential task parallelism to be found within the said algorithm. This is achieved by controlling the granularity of the subtasks comprising the algorithm. At the same time, our solution also endeavors to exploit the inherent data parallelism potentially derived from working with large data sets – whereby one would run the same algorithm multiple times on different data subsets.

The algorithm representation model chosen for our computing platform is a workflow-based one. In this case, the algorithm consists of a series of processing sub-tasks and the connections between them. The sub-tasks, also called operators, are independent functional units capable of performing relatively simple computations. The connections

between them serve to map out the data paths within the overall algorithm – Fig.1. Along with the operator instances and their connection, the model makes use of input and output ports in order to represent the data sources and sinks. Thus, the aim is to represent the flow of data in a simple and intuitive manner, requiring little to no formal training from the user's part [13].

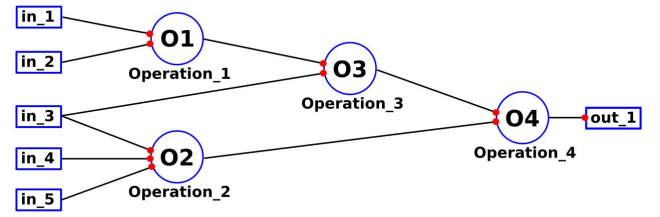


Fig. 1. The basic workflow model

The benefits of the proposed model become apparent when comparing it to the more rigorous and inherently complex mathematical and programming-based representations. Among these benefits, one could count simplicity and modularity. Its simplicity is a basic requirement to allow users to employ it with as little in the way of training as possible. The model does away with the need for programming by relying instead on a set of pre-defined functional units, shared among the users. These units are called operators and they allow the user to build virtually any type of processing task or algorithm. They can be viewed as essentially pre-programmed pieces of functionality ready to be plugged in to create new processes.

Since the availability of different operators might impede the development of certain processing algorithms, one of the most important requirements of the processing platform is to allow for both the easy integration of existing executables into new operators and the reuse of existing operators within new processes. With access to a sufficiently large and diverse library of basic operators, one could develop increasingly complex workflow representations of processing algorithms capable of solving problems from a variety of different domains. To this end, the platform provides the means to easily integrate existing workflows into newer, more complex designs [14]. These workflows can then be shared between different users by simple file transfer of their description, thus encouraging design reuse and reintegration.

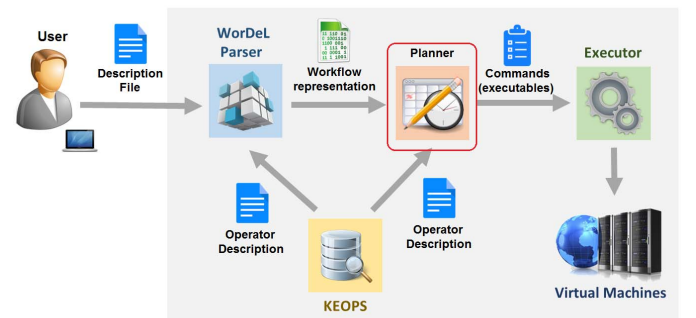


Fig. 2. The BigEarth platform architecture

IV. EXPERIMENTAL SCENE RENDERING

Although it has been designed for the purpose of processing large EO data sets, to prove the flexibility of the BigEarth platform and its ability to take advantage of the available processing power of distributed computing systems, we have decided to test it in a totally different usage scenario, namely the rendering of 3D scenes of objects.

The test infrastructure consists of a collection of virtual machines deployed over a pool of physical resources consisting of 30 virtual CPUs, 125GB of RAM and 1TB of storage. The servers hosting these test resources are located off the university's premises and are hosted by one of our project partners from the city of Cluj-Napoca.

One might notice that there is a serious limitation when it comes to the hardware – namely the lack of graphical processing units (GPU). This means that the actual rendering is done by the CPU. Unfortunately, this is an intrinsic limitation of our computing infrastructure, due to the physical lack of render-capable GPUs. In spite of this, we followed through with the tests since we wanted to prove the system's capability of running parallel rendering operations within a network of worker nodes. Moreover, one should consider the fact that the processing platform is very loosely connected to the actual worker nodes, which are independent machines taking on processing jobs and delivering results. In light of these facts, given access to proper resources – high-performance GPUs and virtualization software, which at the moment is problematic – one could create better performing worker nodes, suitable for the specific task of 3D rendering without incurring any modifications to the platform itself.

Keeping these in mind, the main purpose of our experiments is not to demonstrate the performance of our system in terms of raw processing power, but rather to highlight its flexibility in meeting the demands of different potential users. The tests are also intended to show the system's scalability and the ability to make use of its available computational resources in order to deal with heavy processing loads. In this regard, the problem of rendering complex 3D scenes is ideal, since it provides us and the system with plenty of opportunities for exploiting data parallelism in order to reduce the overall processing time.

A. Infrastructure

For our tests we employed a network of 25 interconnected processing nodes ready to receive tasks dispatched from a couple of orchestration servers across the network. Data files are stored and accessed by the worker nodes from a shared storage element – Fig.3.

The current configuration allows for easy power on and shut down of the worker nodes in order to setup and monitor different task execution loads within the system. Each test case has been run using several system capability levels, defined by the number of the available worker nodes. Each worker node is running Linux and has been assigned one virtual CPU, 2 GB of RAM and 4.80 GB of storage space. In

this configuration the maximum number of worker nodes is only limited by the number of virtual CPU's available. Given the need for a couple of orchestration and shared file system servers, for the purposes of our experiment we will be limiting the number of available workers to 25.

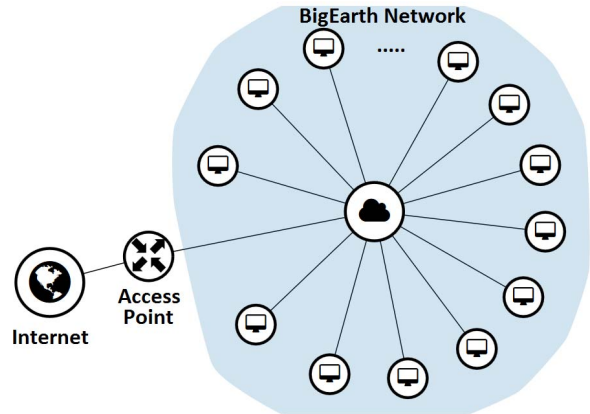


Fig. 3. Network topology

The tests largely follow two directions: the rendering of still frames and the rendering and composition of videos. For these purposes, we have used Blender, a free and open-source 3D creation tool, which provides functionality for the whole range of the 3D pipeline – from mesh modelling and texturing to rigging, rendering and animation [2]. Along with the standard user interface, Blender also offers access to a Python-based API, allowing the user to programmatically access most of its functionality. This feature is at the core of our tests, since it was employed in order to create a small set of operators to use for the required rendering tasks.

Our tests have been performed using Blender's default rendering engine, Cycles, operating on a single 3D scene, namely the Barcelona Pavilion, an architectural visualization demo available for download on the Blender project's website [15]. It is a reasonably complex 3D scene consisting of over 22,000 vertices and various customized shaders for complex effects like glossiness, transparency and refraction – Fig. 4.



Fig. 4. Cycles rendering of the "Barcelona Pavillion" scene (200 samples)

B. Test setup

The first test was intended to ascertain the system’s capability for rendering single frames using the Cycles engine. To this end, we have created a wrapper python script in order to setup the rendering parameters and invoke the engine’s functionality. We have then integrated that script into a frame rendering operator, ready to be plugged into a rendering workflow – Fig. 5.

Since we wanted to partition the task of rendering the frame, we made use of a “for-each” construct – amply described and experimented upon in [16], [17] – in order to create multiple subtasks, operating on the same input files and rendering different regions of the image. To control the region rendering we made use of the various parameters provided by Blender’s Python API. The parameters required by each subtask were distributed through a list of configuration files generated by a special purpose operator, called Frame Split. This is a simple Java program written to take in a rendering configuration file and a number of regions and output a list of configuration files designed to restrict the rendering to predetermined regions, arranged in a simple $n \times n$ grid layout. This list is taken in by the “for-each” node, generating a different rendering subtask for every file within. This is the parallel part of the workflow, as the rendering of one region is completely independent from any other such task. After the completion of all the subtasks, it is also the job of the “for-each” element to aggregate the results into a list, which it then passes to the final node of the workflow, the Image Stitch operator. Again, this is a fairly simple operator, written in Java, whose task is to take all the rendered image fragments and place them into a final image, generating the result of the rendering process.

The Cycles rendering parameters used for this particular test are the following:

- Image resolution: 1920×1080 pixels
- Number of samples: 200
- Max./min. number of bounces: 5/0
- Glossy/volume/transmission bounces: 4/12/1
- Transparent min/max bounces: 8
- Tile size: 32×32

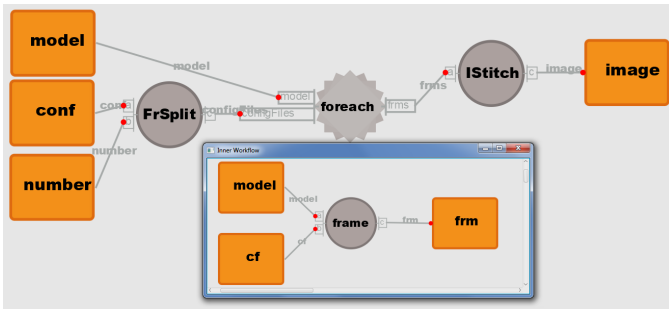


Fig. 5. Single frame rendering workflow (including sub-workflow window)

The second test aims to capture the system’s ability to exploit parallelism at the frame level within a given sequence. For this purpose, we have created a workflow capable of

distributing individual frames and subsequences of frames to different processing subtasks. This workflow is almost identical to the previous one (Fig. 5), except for the first and last operators, responsible with the splitting of input data and the result’s aggregation. In the case of video rendering, we want to split the processing into subtasks dealing with subsets of the total required number of frames. As with the previous test, this is done by a purposefully built operator working on a configuration file to generate a list of files, each designed to force the renderer to only produce one particular frame. This operator is very similar to the frame splitter, the only difference being that it modifies different parameters within the renderer’s configuration file.

The rendering part of the processing is done in the same way, using a “for-each” node to build sub-workflows or subtasks operating with versions of the configuration file which are designed to constrain the rendering to a single, task-specific frame. The result of these tasks takes the form of a list of files, each representing one frame of the future video. Normally, we would have used Blender to also do the final part – creating the video from the frames. However, since we have experienced some troubles with getting the python script to properly create the video, we decided to take a shortcut and instead rely on an open-source, third party tool named FFmpeg [18], which proved to be quite easy to integrate into our Linux working environment. We wrote a simple shell-wrapper based on the executable and integrated it into a video operator, which we then plugged into our workflow in order to generate an “.avi” video file from the list of frames.

V. EXPERIMENT USE CASES

Although not intended to be a measure of the performance potential of our current setup, throughout the tests we will be monitoring the overall execution times of the processes. These are obtained by directly monitoring the database containing the records of each processing workflow. The records contain, among a host of other parameters, the exact starting and ending time of each subtask within a given workflow - with a resolution of 1 second. For the sake of consistency and to smooth out any anomalies, each particular test instance result presented henceforth is computed as the average of three different runs - with the same parameters. To provide the reader with a standard benchmark score for our tests, we have also run each test on a mid-level graphics card – namely the NVidia GTX960 – hosted on a desktop system powered by an Intel i5-4460 CPU with a standard HDD and 16 GB of RAM.

A. Single frame rendering

The first experiment consists of running a single, full-HD, frame rendering task using Cycles with 200 samples. For this test, we chose to run five scenarios using 1, 4, 9, 16 and 25 active workers within the network. During each scenario, we generated a number of image tiles four times larger than the number of available worker nodes. This was done in order to ascertain the degree to which the system would be able to effectively spread and execute the various tasks effectively.

This consideration is rather important, seeing as a rendering task of 200 samples, even at the small resolution of an image tile, is relatively expensive in terms of processing time.

The results of the single frame rendering tests are shown in Fig. 6. In this particular case, we managed to get from a mean of 80 minutes for a 4-tile image run on a single worker to 11.09 minutes for 100 image tiles being processed by 25 workers. Looking at the results, one could notice some significant speedup occurring in the first phases – using four and nine worker nodes. From there on, the speedup obtained slowly decreases. This is partially due to the increasing number of processing tasks (up to 100) per workflow, which tend to generate larger numbers of files. The final speedup limit is also dictated by the processing times of the subtasks. These times tend to vary, since some parts of the image require significantly more processing power to render due to the effects they capture – such as refraction and transparency. Overall, in this example we have managed to get a speedup of 7.23 at the cost of multiplying the processing power by a factor of 25. The best overall time still falls short of the performance obtained from the GTX960. However, in this case, the GPU has the added benefit of not having to access the hard disk to deal with intermediate files and can instead focus strictly on performing scene rendering operations.

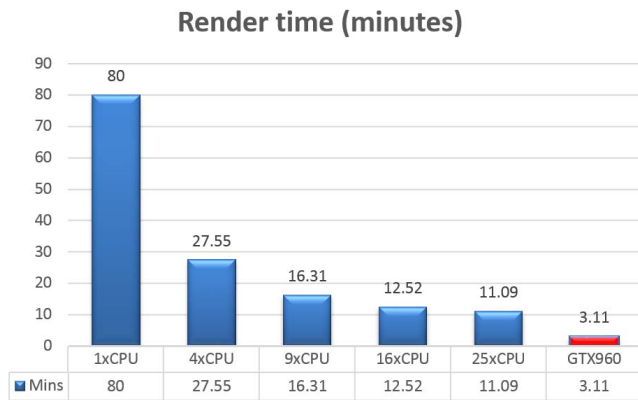


Fig. 6. Single, HD frame rendering times (minutes)

B. Video rendering

The second test has been split into two sub-tests in order to ascertain the system's behavior when dealing with both qualitative and quantitative rendering operations.

The first sub-test aims to measure the system's rendering time for a number of 25 frames at a resolution of 1000×720 and 30 samples per frame. Note that the number of samples is rather low. This is because, given the number of required frames, we had to keep the total processing time within reasonable limits. All the other rendering parameters remained the same as those used in the previous experiment.

The test results are shown in Fig. 7. In this case, we did six different test runs with 4, 8, 12, 16, 20 and 25 active workers. Because of time concerns, we skipped the rendering run with a single active worker. The improvement is noticeable when

comparing the first two runs with 4 and 8 active workers. The second is not really twice as fast, and this is probably due to the fact that we render a total of 25 frames. Since there is little to no variation in frame quality or features, we will consider the rendering times for each frame to be roughly equal. Let us refer to the initial distribution of tasks to all of the available worker nodes as an execution iteration. Having 8 nodes to render 25 frames means that the system will be able to do 24 frames in 3 iterations by issuing tasks to all of the 8 nodes. Obviously, the task is not ideally suited to make the best use out of the 8 nodes, as one of them will have had to render one additional frame while the other 7 would be idle. This is also the case for the 12 worker run, since the task needs three execution iterations to complete. The difference between the 8 and 12 worker runs gives us the rough estimate of the mean execution time for one such iteration, which is equivalent to the rendering of one frame – around 4 minutes. Looking at the 16 worker run – which will be over after two iterations, we get a new estimate of 3.3 minutes. Finally, looking at the difference between the first two runs – requiring 7, respectively 4 iteration, we get a mean of 4.5 minutes per iteration. Assuming an overall mean of 4 minutes per iteration, we can compare it with the total time required by the 25 worker test run. It obviously only requires 1 iteration to complete the job, resulting in a 2.5 minute extra processing time, which could be attributed to the splitting and video composition operators, as well as the time necessary to generate and manage the various intermediate files.

Having said that, it can be seen in the figure that the 25 worker run has been almost two times faster when compared to the GTX 960, which is most likely due to the fact that this time the GPU has to individually create, access and write each generated frame to the hard disk.

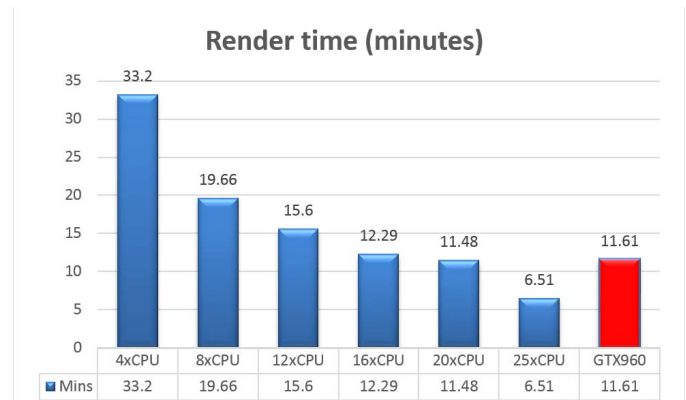


Fig. 7. Rendering times for a series of 25 1000×720 frames (minutes)

For the second sub-test, we decided to render a higher number of frames than the maximum number of available nodes in an attempt to better follow a proper distribution of available tasks within the network. Again, because of time constraints, we have reduced the frame resolution to 800×600 and the quality to just one sample. Of course, one could run the experiment using higher-quality settings, provided access to proper specialized rendering resources for the nodes.

Looking at Fig. 8, one can notice a significantly smoother decrease in the rendering time as we ran the same six scenarios as before. This time, the rendering task consisted of 100 different frames. This resulted in significantly more execution iterations (from 25 down to 4) and therefore gave the system the chance to even out any potential anomalies caused by the occasional fluctuation in frame rendering times or random delays caused by accessing the shared storage.

An interesting aspect to note is that, this time, the GPUs performance matched that of the 25 worker setup. This is probably due to the virtual lack of processing effort spent by the GPU in rendering 800×600 frames at one sample, most of the time being likely occupied by it accessing and writing of the data to the hard disk.

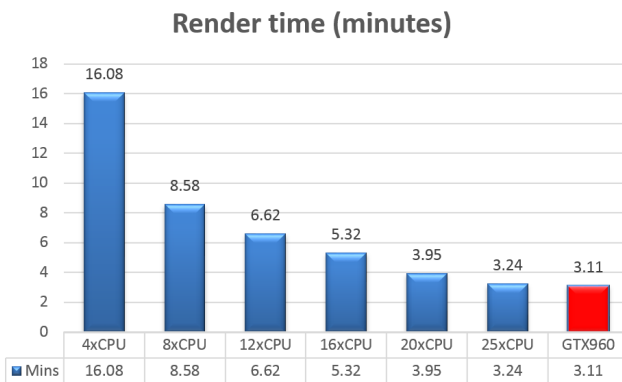


Fig. 8. Rendering times for a series of 100 800×600 frames (minutes)

VI. CONCLUSIONS

As mentioned earlier, the main purpose of these experiments is to prove the viability of using our Big Data processing platform for an entirely different purpose like rendering 3D object scenes. We believe this to be a good measure of the platform’s overall flexibility and adaptability. By showcasing such capabilities, it also reflects its potential usefulness and appeal to new users from other fields of study.

The results presented in the previous section seem to suggest that the system is quite capable of employing its available resources in order to take advantage of inherent data parallelism – provided that the workflow is suitably built by the user. In this case, the use of the “for-each” special construct provided ample opportunities for taking advantage of the data parallelism.

The results also show the system’s tendency towards stabilizing its performance with growing numbers of homogeneous processing tasks. Most importantly, however, the results show the system’s flexibility by proving it can be used for general purpose processing, provided that it is supplied with the proper operator repository. This is indeed one of the system’s main strengths, as well as one of its potential limitations. While it can serve users from virtually any domain, it relies on its user’s willingness to create and

share their designs – workflows and operators – for others to employ and build upon.

ACKNOWLEDGMENTS

The test platform employed for the experiments presented within this paper is part of a research project supported by ROSA (Romanian Space Agency) through Contract CDI-STAR 106/2013, BIGEARTH – Flexible Processing of Big Earth Data over High Performance Computing Architectures. The scientific consultancy and technology transfer has been supported by MEN-UEFISCDI through Contract no. 344/2014, PECSA – Experimental High Performance Computing Platform for Scientific Research and Entrepreneurial Development.

REFERENCES

- [1] Molnar, S., M. Cox, D. Ellsworth, and H. Fuchs. “A Sorting Classification of Parallel Rendering”, in *IEEE Computer Graphics and Algorithms*, pp. 23-32, July 1994.
- [2] Blender.org | Home of the Blender Project, [Online] <https://www.blender.org/>
- [3] Big and Ugly Rendering Project – BURP Home, [Online] burp.boinc.dk
- [4] BOINC – Open source software for volunteer computing, [Online] <https://boinc.berkeley.edu/>
- [5] Flamenco – Open Source Render Manager, [Online] <https://www.flamenco.io/>
- [6] Fox Render Farm | Cloud render Farm, [Online] <http://www.foxrenderfarm.com/>
- [7] Render Rocket | Cloud Farm, [Online] <https://www.renderrocket.com/>
- [8] Rebus Render Farm, [Online] <https://us.rebusfarm.net/en/>
- [9] Zellmann, S., Lang, U. “A Software Architecture for Distributed Volume Rendering on HPC Systems”, in *Proceedings of the LASTED International Conference on Parallel and Distributed Computing and Systems*, November 2012.
- [10] A. Sabou, D. Gorgan, “Remote interactive visualization for particle-based simulations on graphics clusters,” *2017 40th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 2017, pp. 253-258.
- [11] Wu, C., Yang, B., Zhu, W., Zhang, Y. “Toward High Mobile GPU Performance through Collaborative Workload Offloading”, in *IEEE Transactions on Parallel and Distributed Systems*, pp. 435-449, September, 2017.
- [12] BIGEARTH Project Home – <http://cgis.utcluj.ro/projects/bigeearth/>
- [13] V. Bacu, T. Stefanut, D. Gorgan, “Adaptive Processing of Earth Observation Data on Cloud Infrastructures based on Workflow Description”, in *Proceedings of the Intelligent Computer Communication and Processing (ICCP)*, pp.449-454, 2015.
- [14] D. Mihon, V. Bacu, V.D. Colceriu, D. Gorgan, “Modeling of Earth Observation Use Cases through the KEOPS System”, in *Proceedings of the Intelligent Computer Communication and Processing (ICCP)*, pp.455-460, 2015.
- [15] Blender.org | Demo files, [Online] <https://www.blender.org/download/demo-files/>
- [16] C. Nandra, D. Gorgan, “Defining earth data batch processing tasks by means of a flexible workflow description language” in *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. III-4, pp. 59-66, July 2016.
- [17] C. Nandra, V. Bacu, D. Gorgan, “Parallel Earth Data Processing on a Distributed, Cloud Based Computing Architecture”, in *Proceedings of the 21st International Conference on Control Systems and Computer Science (CSCS)*, pp. 677-684, 2017.
- [18] FFmpeg | Video conversion, [Online] <https://www.ffmpeg.org>